DL4Bio 2026

Dr Marcelo A. Marti
Profesor Asociado Dto Química Biológica FCEN-UBA
Inv Ppal del CONICET

Un poco de organización

Cronograma y días de clase
Uso del xls
Tipos de Alumnos
Organización de los trabajo prácticos
Regimen de aprobación

Dudas??

Clase 1: Redes Neuronales

Historia de la IA y aprendizaje automático; neurona artificial, perceptrón y limitaciones lineales (XOR); redes multicapa y funciones de activación; función de pérdida, gradiente y backpropagation como base del entrenamiento en Deep Learning.

¿Por que hablamos de “aprendizaje” o “inteligencia”?

En los albores de la IA (computación) las “computadoras” mostraron ser capaces de resolver problemas intelectualmente difíciles para los humanos (calculo de numeros primos, indexación, factorización), que pueden ser descritos por reglas lógico-matemáticas. El verdadero desafìo para la IA era resolver “tareas” fáciles para los humanos, cómo reconocer “caras” o “voces” que se aprenden naturalmente..

Los pájaros nos inspiraron a volar, las plantas el velcro.

Las neuronas la inteligencia y el aprendizaje.

¿qué significa que un modelo aprenda?

¿qué significa aprender?

Una red neuronal es una función parametrica
Aprender => ajustar parámetros para minimazar una función objetivo

Deep Learning / Redes Neuronales = Prodcuto escalar + No linealidad + Regla de la cadena + Optimización iterativa

Las computadoras eran “buenas” en lo formal y “malas” en lo perceptual
Lo dificil para los humanos era facil para las computadoras y vice-versa

Las Redes neuronales => solucionan ese problema

Perspectiva Histórica

1943 McCulloch y Pitts introducen la neurona artificial (ANN).

1960 primer invierno de las ANN

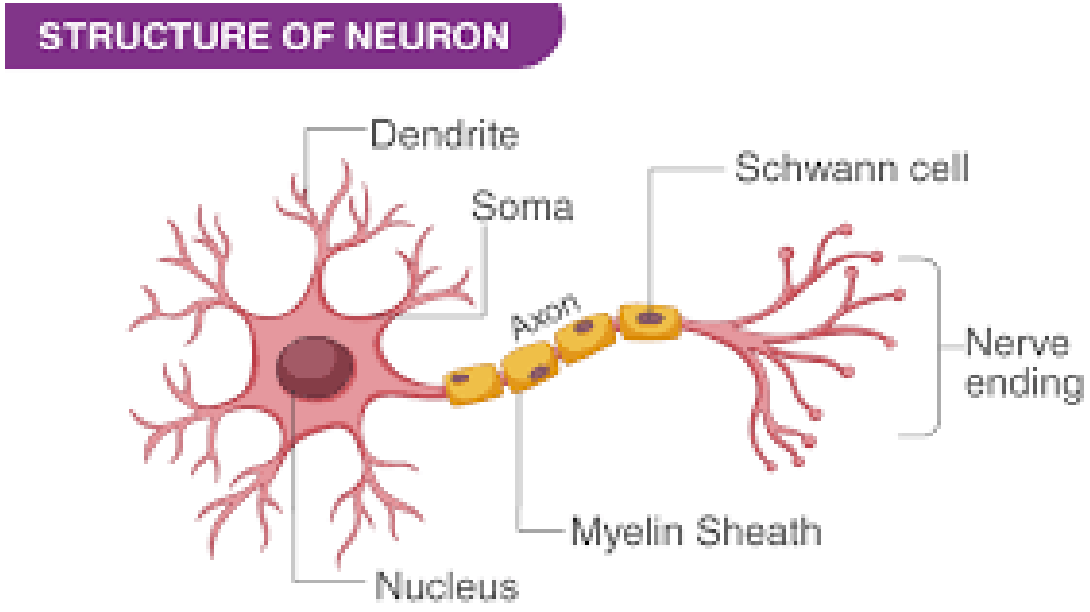
1980 Aparecen nuevas arquitecturas

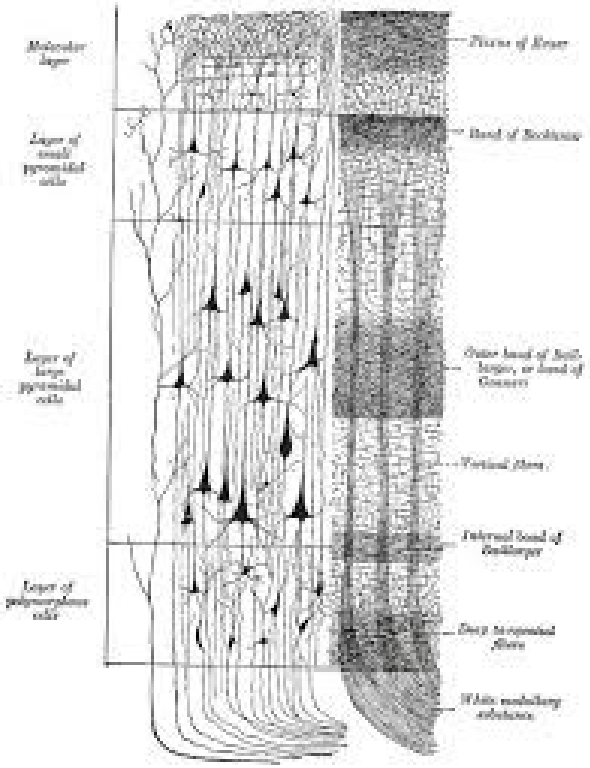
1990 2do invierno de las ANN, por aparición de otros modelos como SVM

2000- Revoluciòn de la IA:

- BigData, GPUs, Modelos de entrenamiento, desarrollos teóricos, ciclo virtuoso de financiamiento-progreso

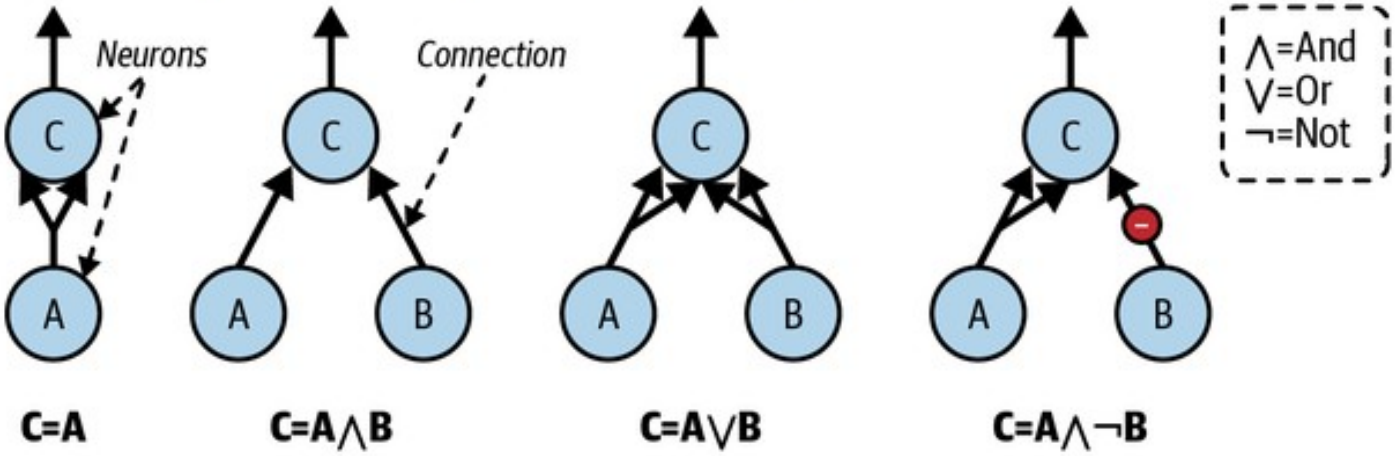
“La(s) Neurona(s)” la inspiración biológica (1943)





Neurona Artificial

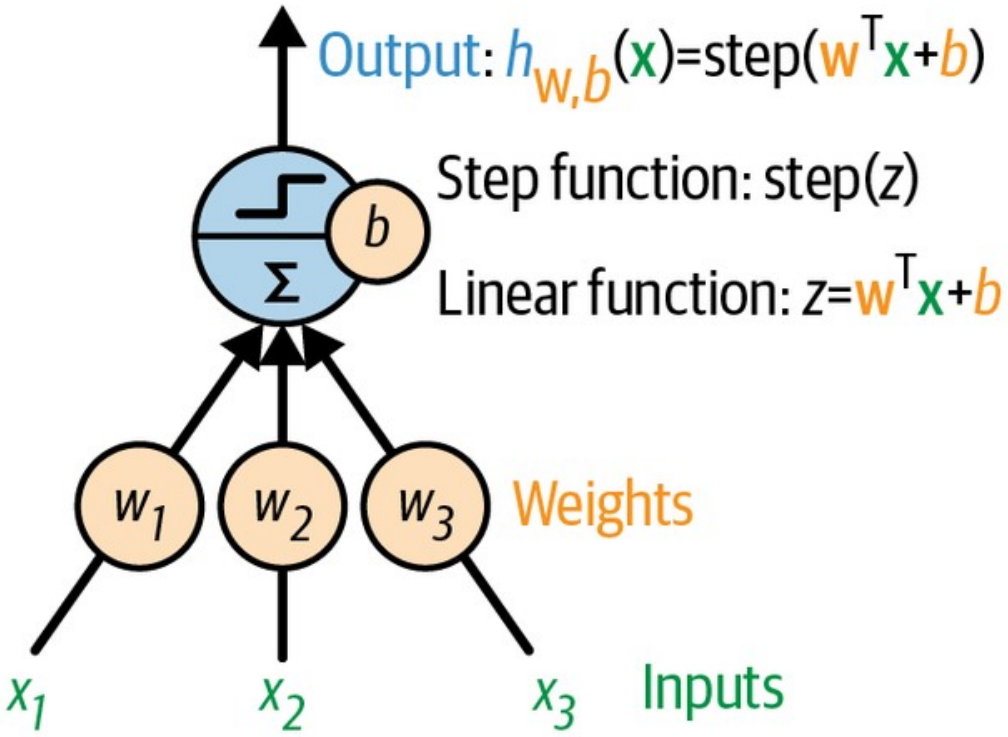
Posee uno (o màs) inputs binarios => output binario que dispara si el input supera un umbral (en el ejemplo si “ambos inputs son +1)



Y que hay del “Exclusive OR (XOR)”?

El “Perceptron” (Rosenblatt 1950/60)

Un perceptron es una NA basada en una TLU (Treshold Logic Unit) que recibe como inputs números(xi), que son combinados linealmente con diferentes “pesos(w)”, un bias (b) y luego una función de activación (i.e step)



El perceptrón

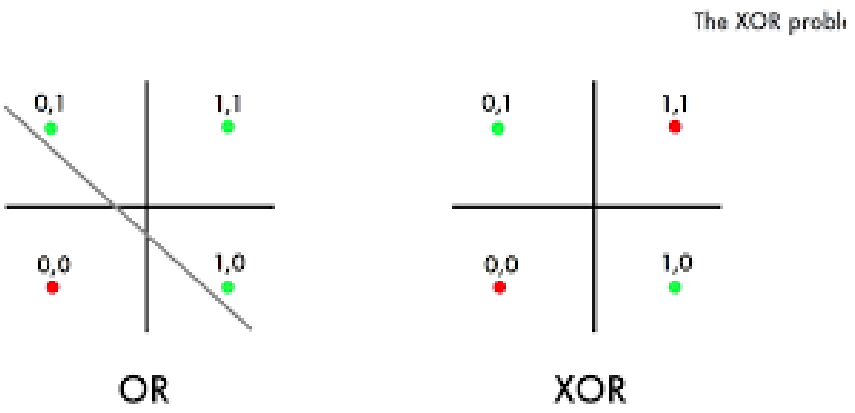
La nuerona tiene pesos ajustables y puede entrenarse

Aparece el concepto: Entrenar =>

Ajustar pesos

El problema del XOR =>

no es linealmente separable



No es un problema de falta de datos sino una limitacion estructural del modelo =>

Primer invierno

El primer invierno, el regreso, 2do invierno

1959: primer invierono se demuestra lsa limitaciones formales del perceptron => la comunidad interpreta que las neuronas artificiales NO sirven

1980: Aparecen los perceptrones multicapa, las capas ocultas, “concepto de profundidad”, regla de la cadena y backpropagation.

Pero...

Pocos datos, poco poder de computo, saturacion de sigmoideas, problemas de gradientes... => segundo invierno

2000s el Renacimiento

Aparecen las GPUs y el Big-Data

Mejores funciones (ReLU)

Formalizacion matematica

red=y=s(Wx+b),

aprendizaje: min L(yp,yR)

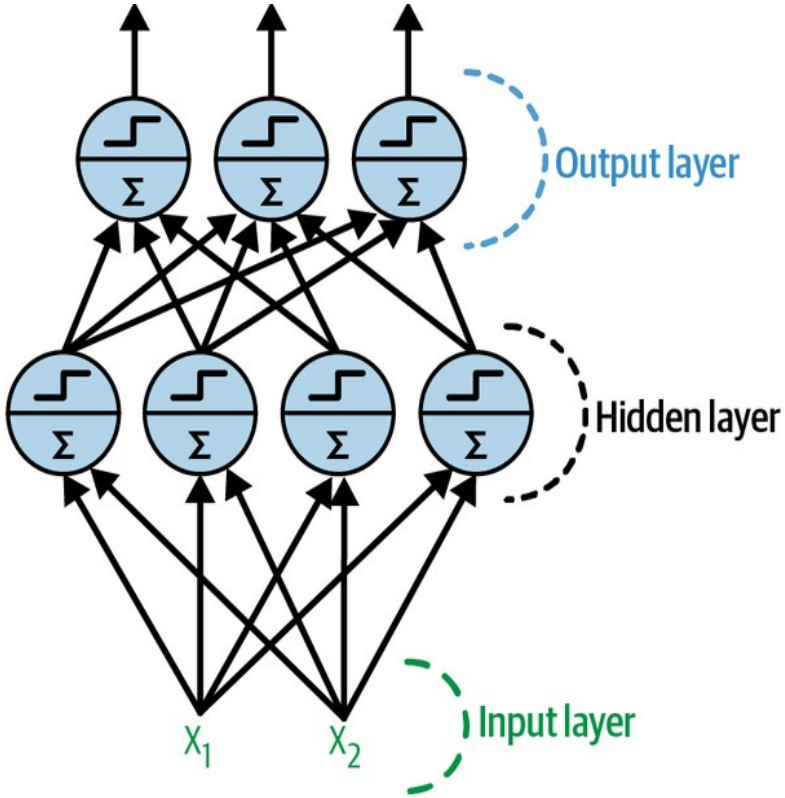
Revolución tecnológica+algorítmica+económica

MutiLayer Perceptron

Los perceptrones aislados tienen la dificultad de que por ejemplo tampoco pueden resolver el XOR, sin embargo muchas de las limitaciones se resuelven “estaqueando” capas sobre capas de perceptrones.

Un MLP que posee muchas capas ocultas (deep Neural Net) => Deep Learning

Ancho vs profundo



El MLP y la solución del XOR

Recordemos que:

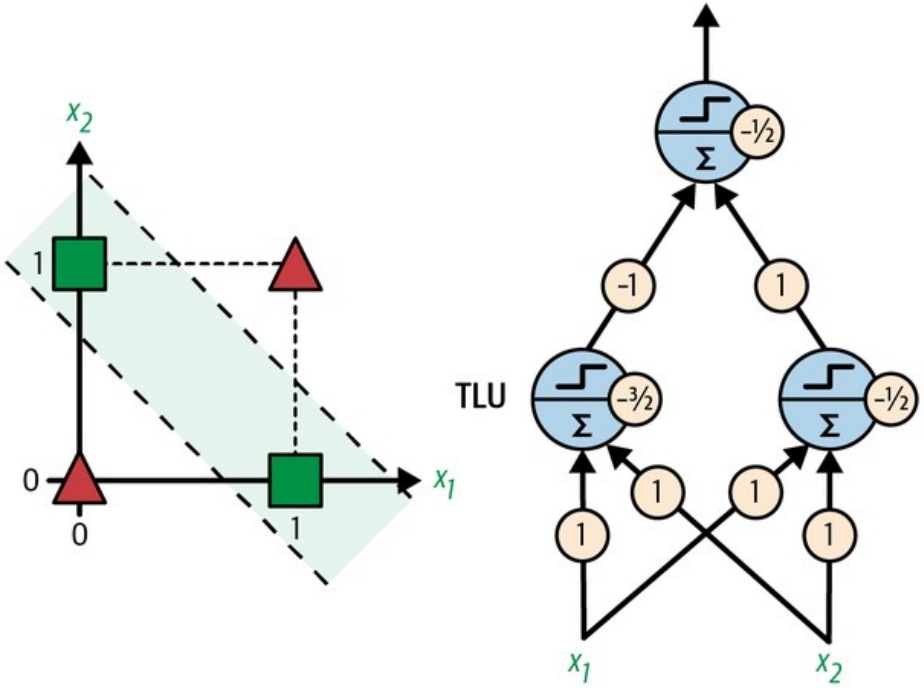
TLU: z = x1+x2-b,

y= 0 (z<0) y 1 (z>0)

XOR:

0,0 / 1,1 => 0

1,0 / 0,1 => 1



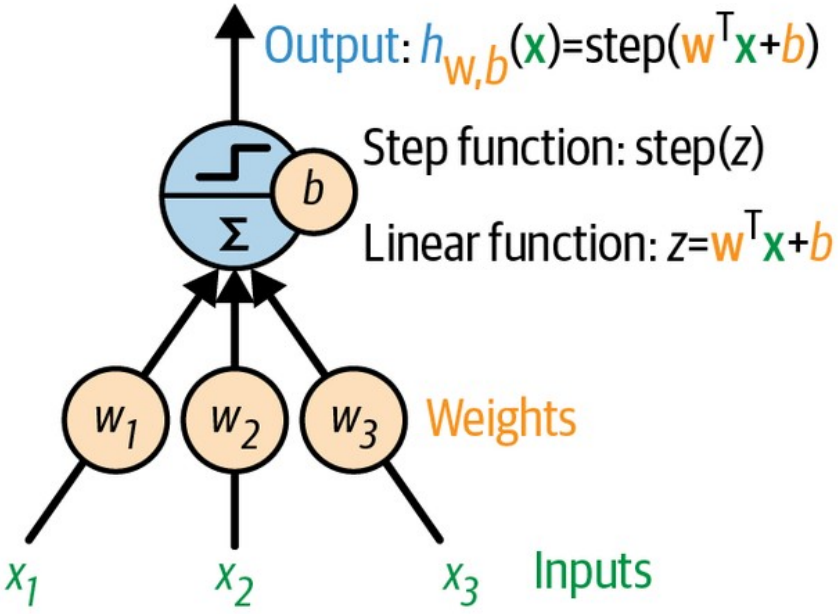
¿Cómo entrenar un MLP?

Entrenar consiste en mover/ajustar los parámetros (wi bi) para que el output sea el deseado

¿Que es el output deseado?

=> Aquel que se parece más al resultado correcto

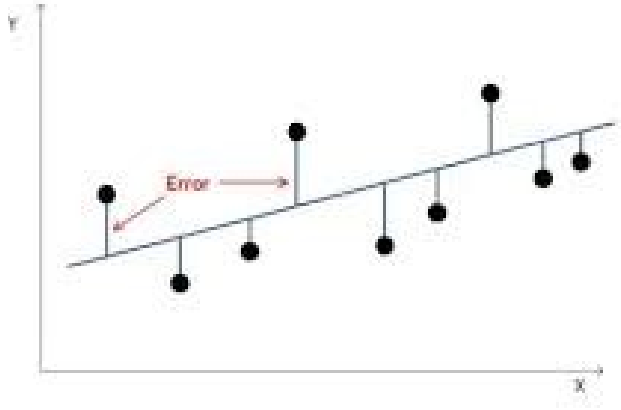
=> Concepto de “loss function”



Loss function (costo) y entrenamiento

Es una métrica que mide qué tan lejos está el modelo de hacer predicciones correctas. Cuantifica el error entre las predicciones del modelo y los valores esperados.

El objetivo del entrenamiento es **minimizar** esta función



$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Entropía Cruzada (Cross Entropy)

Función de pérdida para problemas de clasificación, que la diferencia entre la distribución verdadera de las etiquetas (los valores reales) y las distribuciones de probabilidad predichas por el modelo.

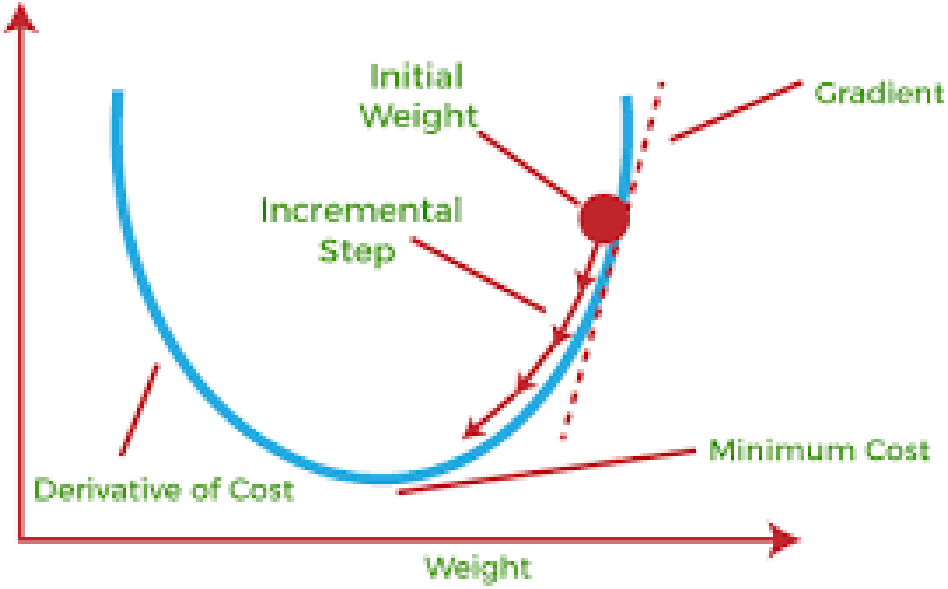
$$\text{Loss} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

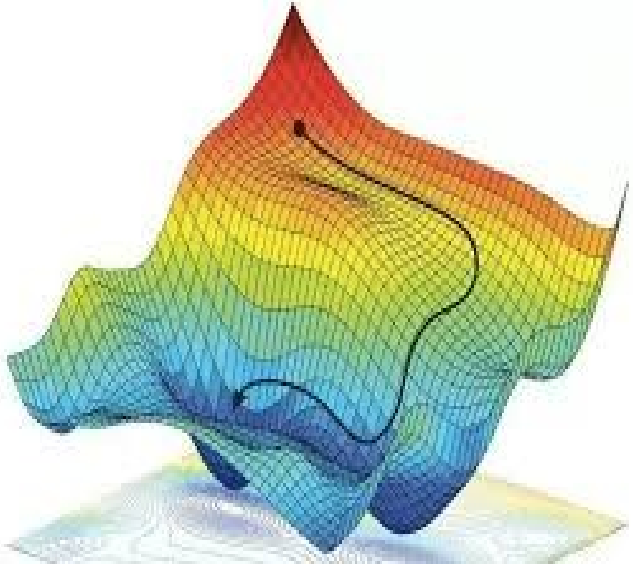
- C: Número de clases .
- yi: Valor real (1 si la clase es correcta, 0 en caso contrario).
- y^i: Probabilidad predicha por el modelo para la clase (log natural).

Cuando el modelo predice probabilidades más cercanas a 1 para la clase -correcta, la pérdida es baja (i.e 1*log(0.9)=-0.1)

Cuando el modelo se "confunde" y asigna una probabilidad alta a una clase incorrecta, la pérdida aumenta rápidamente (log(0.1)=-2.3).

¿Qué es minimizar?





Backpropagation (1.0)

En 1970 “Seppo Linnaianmaa” en su tesis propuso una técnica que permite calcular los gradientes de un MLP de manera automática y eficiente (Reverse Mode Automatic Differentiation)

Para ello se realizan dos “pasos” uno forward de cálculo de predicciones, y luego uno “backward” al calcular los gradientes para cada paràmetro hacia atrás.

Una vez obtenidos los gradientes se utiliza el algoritmo de Gradient Descent para mover los parámetros

W_nuevo = W_viejo - dL/dw *LR (LR = Learning Rate)

El “ciclo” de entrenamiento (la epoca)

1) Se calculan las predicciones para un conjunto /todos los inputs (batch), y se guardan los valores intermedios => Forward pass
2) Se calcula el error (LF) y cuando contribuye cada parámetro (wi, b) de cada capa a la LF (o sea los gradientes) => Backpropagation
3) Se actualizan los parámetros (wi, b) utilizando los gradientes (minimización)
4) Se comienza un nuevo ciclo...

Es clave que los paràmetros estén inicializados con valores (i.e al azar) y no sean cero.

¿como se que la red se está entrenando?

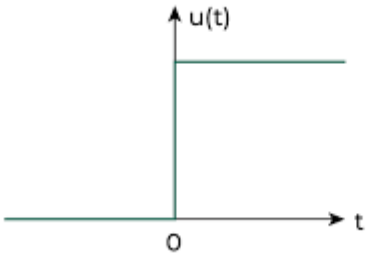
La LF se hace cada vez más pequeña

Las métricas de capacidad predictiva (Accuracy, TPR etc.) mejoran tanto en el set de entrenamiento como validación

Mejora la forma de las curvas ROC y PR

La función de activación y los gradientes

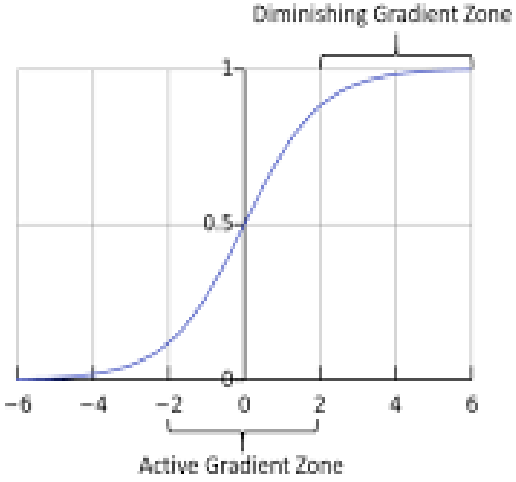
¿cuàl es la derivada de la función escalón?



$$u(t) = \begin{cases} 1 & t > 0 \\ 0 & t > 0 \end{cases}$$

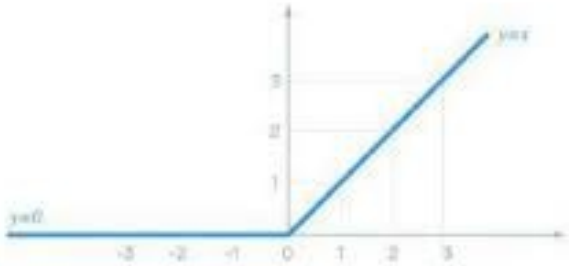
Sigmoidea

$$A = \frac{1}{1+e^{-x}}$$



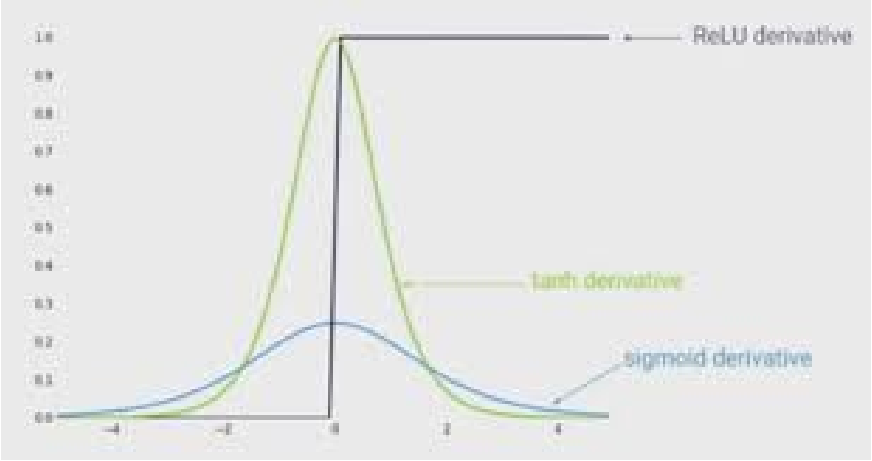
ReLu

$$f(x) = max(0, x)$$



¿ReLu contribuyo al renacimiento de las RN?

Porque evita el problema de saturación de gradientes que tenían sigmoide y tanh. ReLU es lineal para x>0, por lo que su derivada no se hace pequeña y el gradiente puede propagarse a través de muchas capas, permitiendo entrenar redes profundas de forma estable y eficiente.



¿por qué se necesita una función de activación (FA)?

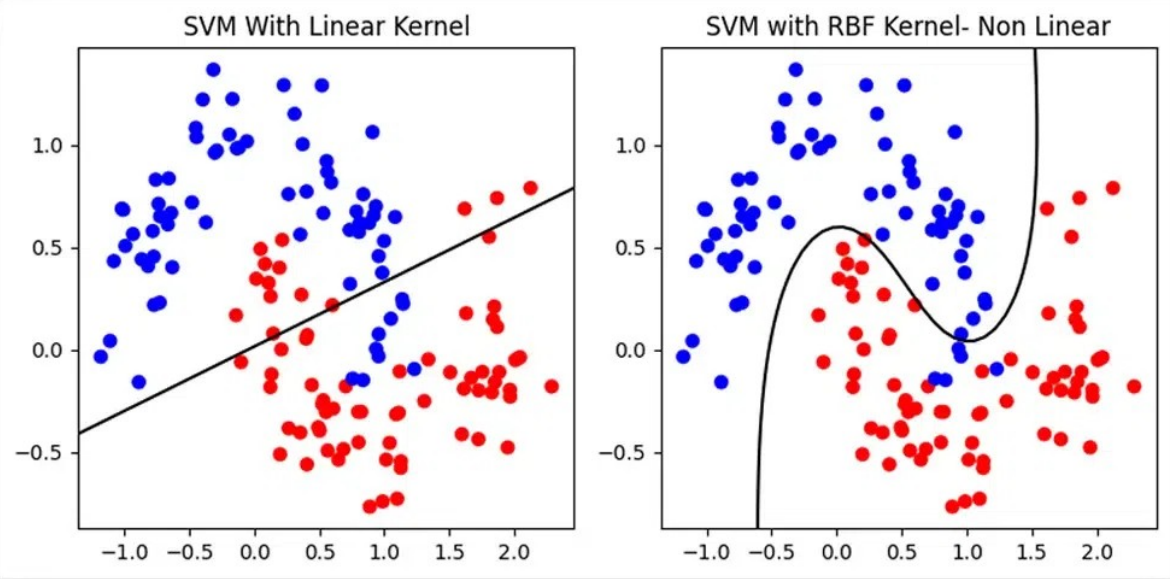
La FA es la que otorga la NO-linealidad.

Un MLP con 1 capa oculta => aproximador universal (puede replicar cualquier función)

Precisión es proporcional al “ancho” de la capa

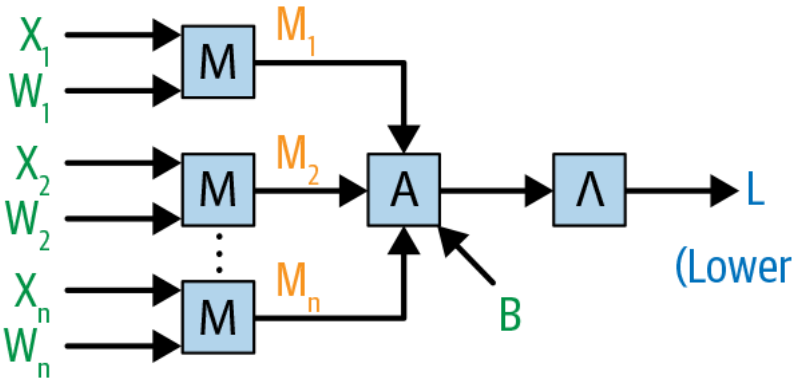
Los modelo profundos (muchas capas) son más eficientes que los modelos anchos..

Comparando un problema de clasificación

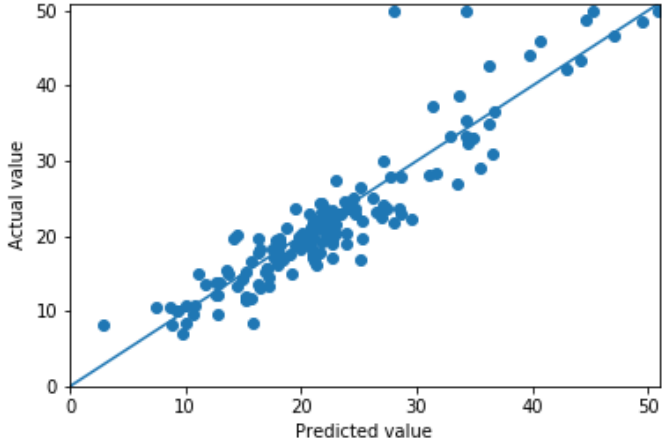


Evidencia de “no-linealidad”.

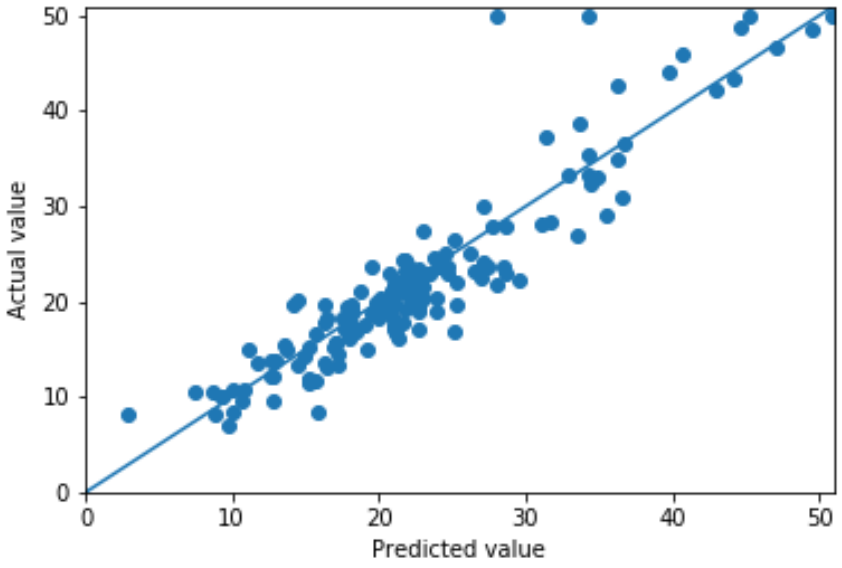
Comparemos para modelo de regresión, una regresión lineal múltiple con una pequeña red neuronal



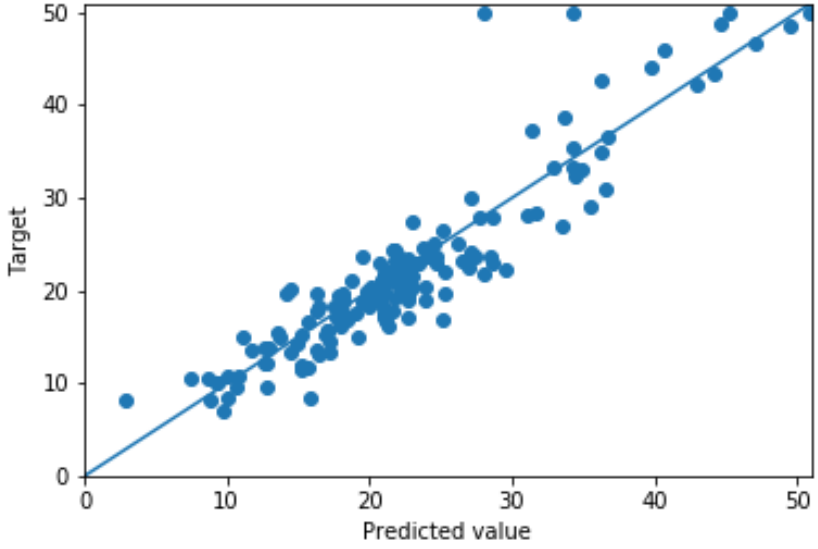
En un modelo de regresión LF =RMSE y la manera gràfica de analizar la performance es graficando y_pred vs y_real



RLM vs RN

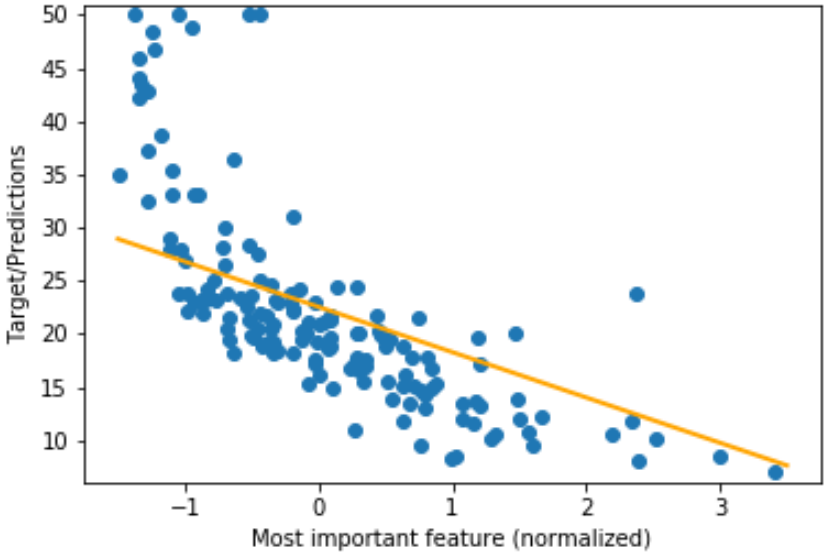


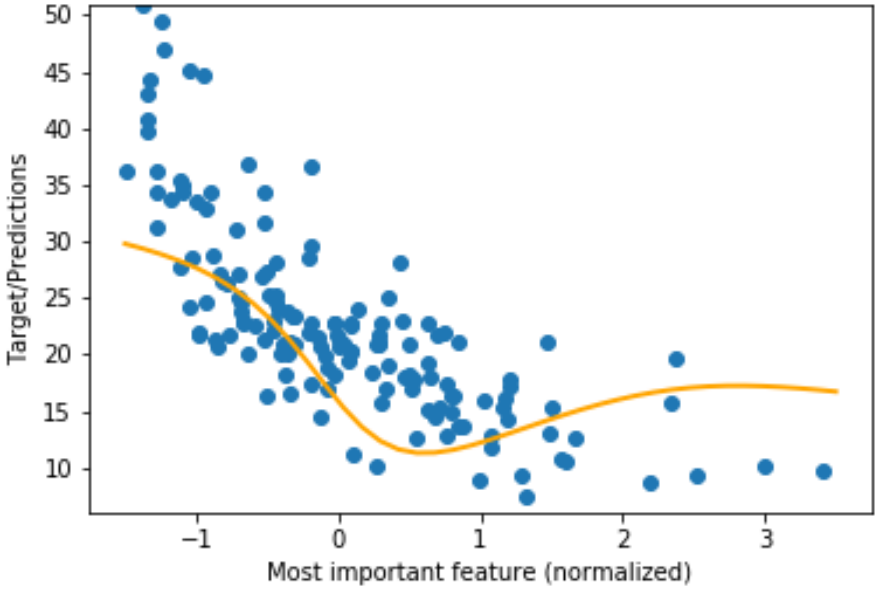
Mean absolute error: 3.5643
Root mean squared error: 5.0508



Mean absolute error: 2.5289
Root mean squared error: 3.6775

Análisis de contribuciones del “predictor” más importante





RLM vs RN

Salidas para regresión y clasificación

Supongamos que tenemos una red neuronal, cuya última capa es “una (1)” neurona con FA:

a) Lineal

b) ReLu

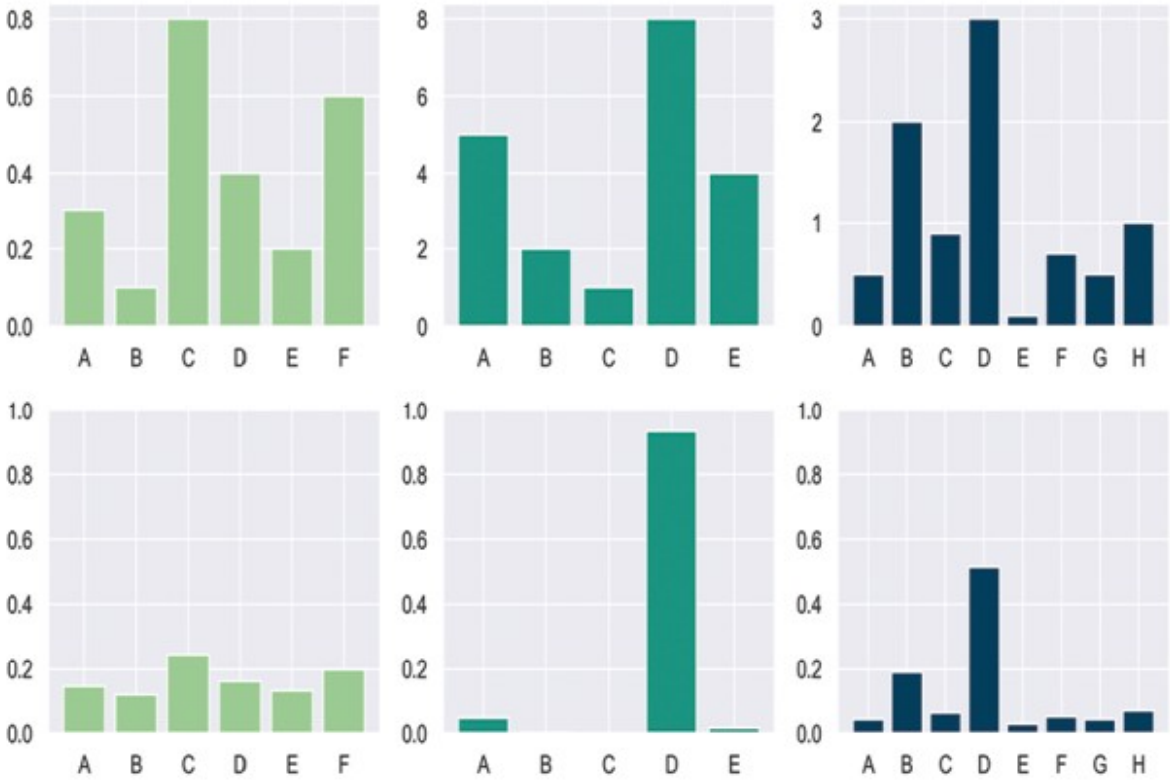
c) Sigmoidea

¿cuál el es el posible output?

¿cómo lo puedo adaptar a una clasificación de 3, o màs categorías?

Softmax

Softmax convierte un vector de números en una distribución de probabilidades, asignando valores entre 0 y 1 cuya suma es 1. Lo hace exponenciando cada valor y normalizándolo dividiendo entre la suma de todas las exponenciales



De la sigmoidea a softmax

Por qué Softmax produce una distribución:

La última capa de la RN produce valores arbitrarios llamados logits z1,z2,...,zk =>

Softmax los transforma en probabilidades:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Por lo tanto, el vector de salida es una distribución de probabilidad sobre las clases.

Preguntas... (para ver si entendieron)

¿por qué XOR necesita capa oculta?

¿qué es un gradiente?

¿qué minimizamos?

¿Escribir forward/backward?

¿Pueden distinguir arquitectura de entrenamiento?

TP1: De la Neurona al DL

Objetivo: Conocer y programar la estructura bàsica de una neurona artificial, y un perceptron multicapa. Resolver con pequeñas redes neuronales problemas de regresión y clasificaciòn simples. Fortalecer los conceptos de forward y backward pass, y de backprogation. Realizar un ciclo de aprendizaje y de anàlisis de resultados.

1) ¿Programar una (1) Neurona?
2) Puedo con una neurona resolver AND, OR, XOR?

El Backprop

Programar “Backpropagation”

¿que elementos tiene?

¿como podemos testear que los módulos funcionan?

Ejercicio 1.1: Entrenando el AND

Utilizando una neurona de dos entradas (x1, x2) y la función de activacion sigmoidea, programe un código que tomando valores de w1,w2 y b de entrada al azar, determine los gradientes y los vaya actualizando hasta encontrar un valor óptimo tal que la neurona resuelva el AND.

Controle del mismo la cantidad de ciclos de entrenamiento, y el LR.

Para monitorear el entrenamiento calcule (y grafique) el avance de la LF en función del número de ciclo y calcule y muestre los valores finales que obtiene la neurona para cada una de las 4 condiciones de entrada del AND.
Analice los resultados

b) Que pasa si en lugar de una sigmoidea utiliza ReLu?

c) Se anima a repetir el proceso pero para la función lógica “OR”

d) ¿que pasa si intenta resolver el “XOR”?